Introduction to Robotics, Lecture 8

Wheel constraints and holonomy

Polar feedback control

Global and local navigation

MIPT

Center for Cognitive Modeling

Where we stopped, and why we go back

From one wheel to a useful base

W07 ended at manoeuvrability. We revisit the constraints slowly, using differential drive, three-Swedish omni, and differential drive with a castor.

Three properties to read

A **constraint** forbids a motion. **Mobility** counts independent instantaneous velocities. **Steerability** counts independent steering changes.

An **instantaneous centre of rotation (ICR)** is the point about which the chassis turns at an instant. One wheel-by-wheel recipe lets us compare the bases.

One method: constrain each wheel, then stack

1. Constrain each wheel

Rolling: speed along its driven direction matches wheel spin. Sliding: no speed across a standard wheel.

2. Stack the rows

$$J_1(\beta_s)\dot{\xi}_R = J_2\dot{\varphi}, \qquad C_1(\beta_s)\dot{\xi}_R = 0.$$

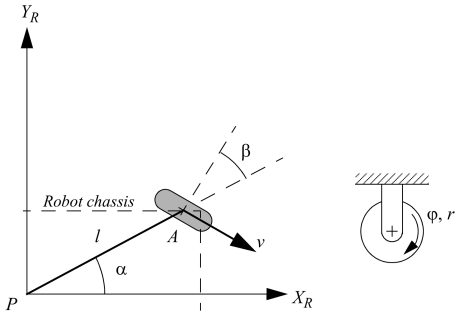
Read the symbols

$\xi = (x, y, \theta)^T$: planar pose. $\dot{\xi}_R = (\dot{x}_R, \dot{y}_R, \dot{\theta})^T$: velocity in the robot coordinate frame. $\dot{\xi}_R = R(\theta)\dot{\xi}_I$, where R maps global velocities to robot coordinates. $\dot{\varphi}$: wheel spin rates. β_s: steerable-wheel angles.

3. Read the mobility

J_1: rolling rows; J_2: wheel-radius factors; C_1: sliding rows. Rank counts independent forbidden directions. $\delta_m = 3 - \operatorname{rank} C_1$ is mobility.

A fixed wheel allows rolling and forbids sliding



l: centre-to-wheel distance; α: polar position angle; β: plane orientation; r: radius; φ: spin angle.

Rolling without slip

$$\sin(\alpha+\beta)\dot{x}_R - \cos(\alpha+\beta)\dot{y}_R - l\cos\beta\,\dot{\theta} = r\dot{\varphi}.$$

Contact speed along the wheel plane matches rim speed.

No sideways sliding

$$\cos(\alpha+\beta)\dot{x}_R + \sin(\alpha+\beta)\dot{y}_R + l\sin\beta\,\dot{\theta} = 0.$$

Contact speed across the plane is zero.

Worked example: one fixed wheel

Substitute the geometry

$\alpha = 90°$, $\beta = 0$, $l = 0.2$ m, $r = 0.05$ m, $\theta = 0$. Global and robot coordinates coincide.

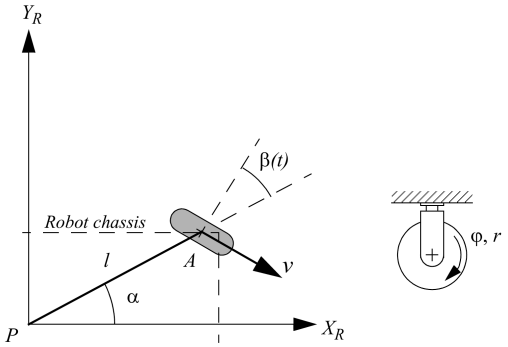
$$\underbrace{\begin{bmatrix} 1 & 0 & -l \end{bmatrix} \dot{\xi}_R = r\dot{\varphi}}_{\text{rolling}} \qquad \underbrace{\begin{bmatrix} 0 & 1 & 0 \end{bmatrix} \dot{\xi}_R = 0}_{\text{sliding}}$$

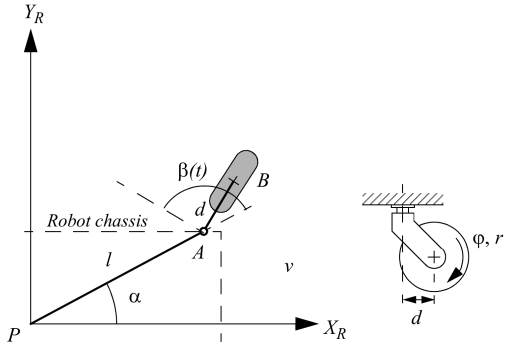
The two measurable statements

$$\dot{x}_R - 0.2\dot{\theta} = 0.05\dot{\varphi}, \qquad \dot{y}_R = 0.$$

If $\dot{\varphi} = 10$ rad/s and $\dot{\theta} = 0$, the robot moves forward at 0.5 m/s and cannot move sideways.

Steered wheel and castor wheel

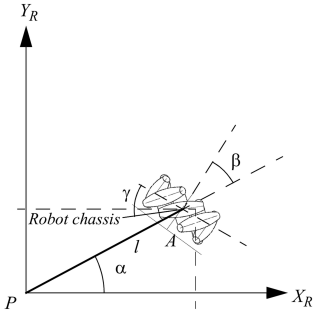




Steered: choose $\beta(t)$, the wheel-plane angle. Same rolling/sliding rows.

Castor: offset d adds $d\dot{\beta}$. Free pivot: no chassis constraint, only support.

Swedish wheel: rollers remove the sliding constraint



A **roller** is a freely spinning small rim wheel. γ is the angle between its axis and the main wheel plane. Roller spin leaves one contact direction free.

$\gamma = 0$: Swedish 90° wheel. The standard rolling row remains; no lateral sliding row. Driven and free directions decouple.

$\gamma = \pi/2$: degenerate and excluded. Lateral constraint remains, rolling constraint disappears; main-wheel spin is unnecessary.

Stacking: J1, J2 and C1

Rolling rows collect the wheel speeds

$$J_1(\beta_s)\dot{\xi}_R = J_2\dot{\varphi}, \qquad J_2 = \operatorname{diag}(r_1, \ldots, r_n) \quad \text{(standard wheels)}.$$

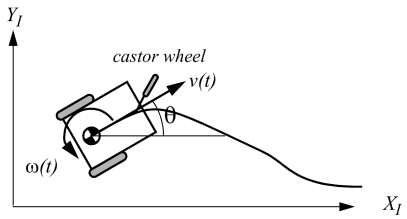
A Swedish row instead has $r \cos\gamma\, \dot{\varphi}$ on the right. Our omni uses $\gamma = 0$, so its factor is simply r.

Sliding rows collect the forbidden velocities

$$C_1(\beta_s)\dot{\xi}_R = 0, \qquad \delta_m = 3 - \operatorname{rank} C_1.$$

Only fixed and steered standard wheels add C_1 rows; dependent rows count once. Castor, Swedish and spherical wheels add none.

Differential drive from the recipe



Two fixed wheels, radius r, axle half-width l.

$$J_1 = \begin{bmatrix} 1 & 0 & l \\ 1 & 0 & -l \end{bmatrix}, \quad C_1 = \begin{bmatrix} 0 & 1 & 0 \end{bmatrix}.$$

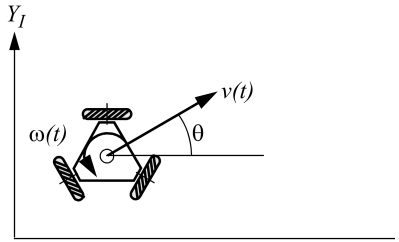
$$\dot{x}_R = \frac{r}{2}(\dot{\varphi}_1 + \dot{\varphi}_2), \quad \dot{\theta} = \frac{r}{2l}(\dot{\varphi}_1 - \dot{\varphi}_2).$$

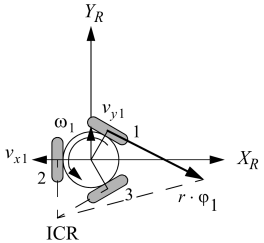
Wheel labels follow the displayed rolling rows; positive spin drives forward.

Mobility

$\dot{y}_R = 0$; $\operatorname{rank} C_1 = 1$; $\delta_m = 2$. Translation and rotation remain independent.

Three Swedish wheels: no constraints at all

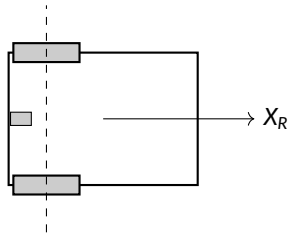




$\gamma = 0$, 120° spacing; C_1 empty, $\delta_m = 3$. Any planar velocity: $\dot{\xi}_R = J_1^{-1} r\dot{\varphi}$.

$$J_1 = \begin{bmatrix} \sqrt{3}/2 & -1/2 & -l \\ 0 & 1 & -l \\ -\sqrt{3}/2 & -1/2 & -l \end{bmatrix}, \qquad J_1^{-1} = \begin{bmatrix} 1/\sqrt{3} & 0 & -1/\sqrt{3} \\ -1/3 & 2/3 & -1/3 \\ -1/(3l) & -1/(3l) & -1/(3l) \end{bmatrix}.$$

Differential drive plus a castor: nothing changes



Two drive wheels + one free castor

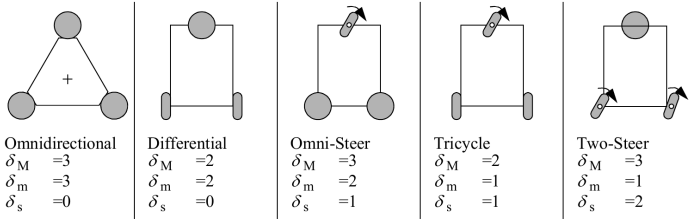
Support changes; mobility stays

The castor adds wheel-spin and pivot variables, but no row to C_1. Thus $\operatorname{rank} C_1 = 1$ and $\delta_m = 2$, just as before.

A fixed third wheel is different

If its sliding row is independent, it further restricts the chassis. A suitably aligned redundant wheel adds no independent row.

Mobility, steerability and manoeuvrability



Base	rank C_1	δ_m	δ_s	δ_M	Sideways
Differential	1	2	0	2	no
Diff. + castor	1	2	0	2	no
Swedish	0	3	0	3	yes

$\delta_m = 3 - \operatorname{rank} C_1$: independent velocities.

δ_s: independent steering sets.

$\delta_M = \delta_m + \delta_s$: manoeuvrability.

Two-steer: $\delta_M = 3$, not omni.

The reason is the instantaneous centre of rotation

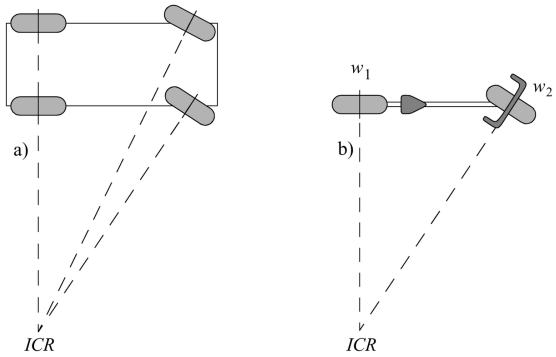
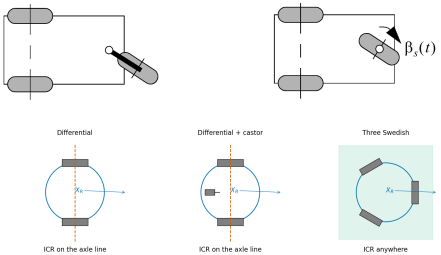


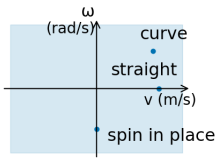
Figure 3.12
(a) Four-wheel with carlike Ackerman steering. (b) Bicycle.



ICR: instantaneous centre of rotation.
Fixed wheels require it on their axle line; omni permits it anywhere.
Straight travel: ICR at infinity. Rest: no unique ICR.

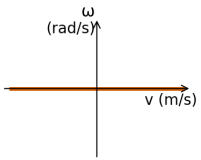
Control freedoms versus freedoms in the workspace

Differential drive
DDOF = 2



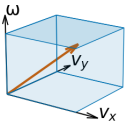
Two independent velocities

Bicycle
DDOF = 1



One independent velocity
Steering angle sets ω/v
at fixed steering

Omnidirectional
DDOF = 3



Three independent velocities

DDOF: independent instantaneous velocities = δ_m (differential degrees of freedom). **DOF**: dimension of reachable chassis poses. $\mathrm{DDOF} \leq \delta_M \leq \mathrm{DOF}$.

Bicycle: (1, 2, 3); omni: (3, 3, 3). Differential / unlocked Ackermann: DOF = 3.

Configuration matters: synchro drive DOF = 2; locked bicycle DOF = 1.

Holonomic and nonholonomic constraints

Holonomic: a position rule

A constraint is **holonomic** if it can be written using positions only. **Integrable** means a velocity rule is secretly a rule about position.

Nonholonomic: a velocity rule that cannot be integrated

A fixed wheel forbids sideways velocity, not sideways displacement reached by manoeuvring.

$$-\sin\theta\,\dot{x} + \cos\theta\,\dot{y} = 0.$$

Turning, driving, then turning can reach a sideways pose. The sliding rule is therefore nonholonomic; it cannot be replaced by one position-only condition for the mobile base.

Locked bicycle, and the DDOF = DOF test

DDOF: differential degrees of freedom = independent instantaneous velocities the robot can set = δ_m. **DOF**: dimension of reachable poses.

Lock the bicycle steering straight

Two parallel fixed wheels lie one behind the other. With the initial line chosen as X, the constraints become $y = 0$, $\theta = 0$; rolling gives $\varphi = x/r + \varphi_0$. Here φ_0 is initial wheel rotation.

All restrictions are integrable. Only translation along the line remains.

A clean chassis-level test

$$\text{holonomic} \iff \text{DDOF} = \text{DOF}.$$

Locked bicycle: $1 = 1$, holonomic but not omni.

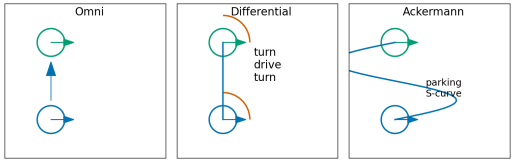
Omnidirectional

Holonomic with DDOF $= 3$: arbitrary planar translation and rotation are immediately available.

Our bases on the holonomic scale

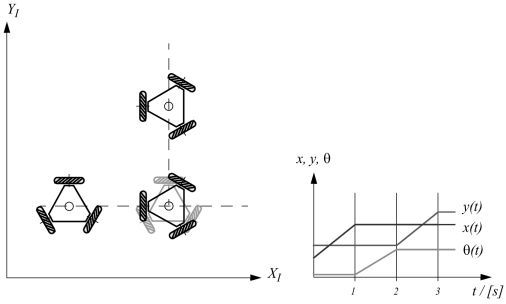
Base	DDOF	DOF	Holo.	Omni
Differential	2	3	no	no
Diff. + castor	2	3	no	no
Swedish omni	3	3	yes	yes
Ackermann	1	3	no	no

More instantaneous freedoms mean fewer preparatory manoeuvres.



Same sideways displacement, different motion sequences.

Path versus trajectory



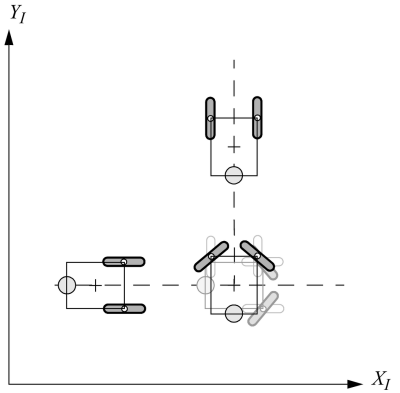
Path and trajectory

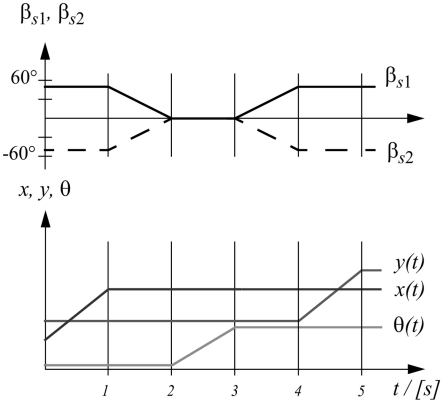
A **path** is a trace in pose space (x, y, θ). A **trajectory** adds time: (x, y, θ, t).

Three phases: 1 s along X at 1 m/s; 1 s rotating by 90°; 1 s along Y at 1 m/s.

An omni can follow this ideal trajectory exactly by changing wheel speeds at the switches. Infinite acceleration is assumed.

Two-steer cannot follow the same trajectory





$\delta_M = 3$: same path, but steering takes time. Three motion phases + two steering phases = five.

Why not omni everywhere, and what kinematics leaves out

Omni has physical costs

Roller wheels are complex and expensive. Ground clearance and suspension are harder to provide.

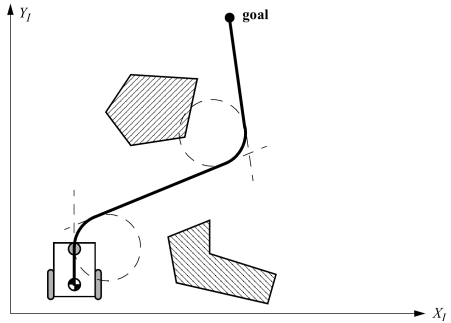
In a fast curve, car-like sliding constraints carry lateral force passively. An omni must supply lateral force through motor torque; it may be thrown out of the curve.

Kinematics describes possible velocities

Dynamics adds mass, inertia and force. **Motorization** concerns the forces and torques the motors must supply. Controllability asks whether the available commands can produce the required motion.

A feasible geometric path still needs a controller that can execute it.

Open loop: lines and arcs, and why it is not enough

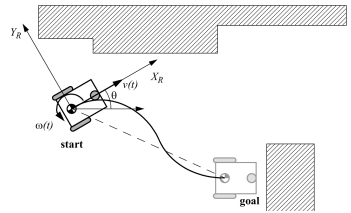


Open loop: precompute straight segments and arcs; do not feed measured pose back. A segment is one piece of the route.

Three weaknesses:

1. Feasibility is hard to precompute with speed and acceleration limits.
2. No adaptation to environment changes.
3. Nonsmooth transitions cause acceleration jumps.

Feedback control: the pose error and why a gain matrix fails



$e = (\Delta x, \Delta y, \Delta\theta)^T$: pose error in the robot coordinate frame. A 2×3 gain matrix K suggests $(v, \omega)^T = Ke$.

v: forward speed; ω: yaw rate.

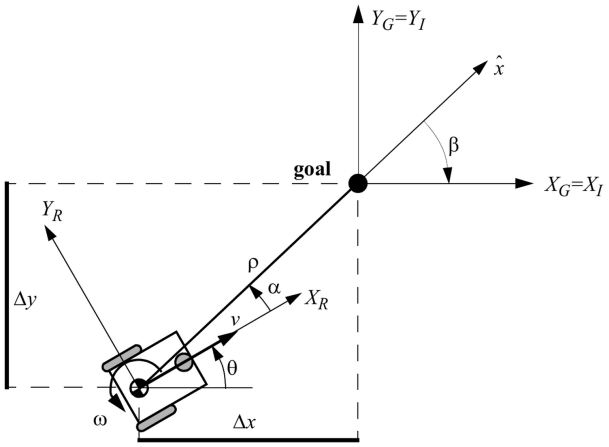
$$\dot{x} = v\cos\theta, \quad \dot{y} = v\sin\theta, \quad \dot{\theta} = \omega.$$

Two inputs control three pose outputs. Sideways error first needs a change in heading.

The smooth-feedback obstruction

No smooth time-invariant state feedback defined around the goal asymptotically stabilises the full pose. Polar feedback avoids this restriction because its Cartesian form is undefined at $\rho = 0$.

Polar coordinates around the goal



Goal at origin, heading 0; robot heading θ.

Global displacements:

$\Delta x = x_g - x$, $\Delta y = y_g - y$.

$\rho = \sqrt{\Delta x^2 + \Delta y^2}$:

distance to goal.

$\lambda = \mathrm{atan2}(\Delta y, \Delta x)$:

goal bearing.

$\alpha = \mathrm{wrap}(\lambda - \theta)$:

heading to goal line.

$\beta = \mathrm{wrap}(-\theta - \alpha)$:

goal line to goal heading.

Wrap angles to $(-\pi, \pi]$.

The model in polar coordinates, and its limits

Forward: goal in front

$$\dot{\rho} = -v\cos\alpha,$$
$$\dot{\alpha} = v\sin\alpha/\rho - \omega,$$
$$\dot{\beta} = -v\sin\alpha/\rho.$$

$I_1 = (-\pi/2, \pi/2]$ is the forward sector. Choose direction at $t = 0$, once.

Reverse: physical $v = -v_c$**,** $v_c \geq 0$

$$\dot{\rho} = v_c\cos\alpha,$$
$$\dot{\alpha} = -v_c\sin\alpha/\rho - \omega,$$
$$\dot{\beta} = v_c\sin\alpha/\rho.$$

For a goal behind, flip both headings:

$$\theta\prime = \theta + \pi, \qquad \phi\prime = \phi + \pi.$$

ϕ is requested goal heading; positions stay fixed.

$$\alpha\prime = \mathrm{wrap}(\alpha - \pi),$$
$$\beta\prime = \mathrm{wrap}(\beta + \pi).$$

Use the forward law on primed errors; physical commands are $(-v_c, \omega_c)$. Same relative heading: requested pose, rear-first.

At $\rho = 0$ the transform is undefined. Stop on position *and* heading tolerance.

The control law and the closed loop

Forward-mode feedback

$$v = k_\rho \rho, \qquad \omega = k_\alpha \alpha + k_\beta \beta.$$

A gain k sets reaction strength.

Distance sets speed; α turns toward the goal; β aligns final heading.

Reverse: same law on primed errors, physical $v = -k_\rho \rho$.

Closed loop

$$\dot{\rho} = -k_\rho \rho \cos \alpha,$$
$$\dot{\alpha} = k_\rho \sin \alpha - k_\alpha \alpha - k_\beta \beta,$$
$$\dot{\beta} = -k_\rho \sin \alpha.$$

No singular term remains in these equations. With stabilising gains, the goal equilibrium is $(\rho, \alpha, \beta) = (0, 0, 0)$.

Stability conditions and how to read the gains

Local exponential stability

$$k_\rho > 0, \qquad k_\beta < 0, \qquad k_\alpha > k_\rho.$$

Sufficient sector condition: retain the chosen direction

$$k_\alpha > \frac{2k_\rho}{\pi} + 2|k_\beta|.$$

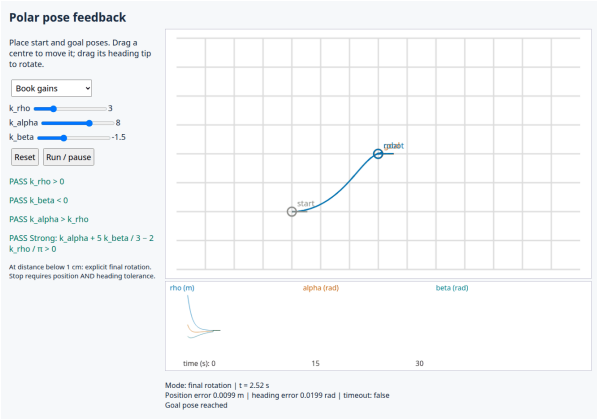
With local conditions: inward $\dot{\alpha}$ at $\alpha = \pm\pi/2$ for every wrapped β; primed in reverse.

$$\dot{\alpha}|_{\alpha=\pi/2} \leq k_\rho + |k_\beta|\pi - k_\alpha\pi/2 < 0.$$

Intuition: k_ρ sets speed; too large exceeds wheel limits. $k_\alpha > k_\rho$ turns faster than approach. $k_\beta < 0$ aligns final heading.

Book gains $(3, 8, -1.5)$: $2k_\rho/\pi + 2|k_\beta| \approx 4.91 < 8$.

Demo 1: choose the gains yourself



Book gains (3, 8, −1.5); vary gains and watch position, final heading and stability. pixi run serve.

What this controller does not know: pose stabiliser versus path tracker

Pose stabiliser

One requested goal pose. Differential-drive kinematics. No obstacles or acceleration limits in the model.

It reaches a pose without deciding which side of a wall to use.

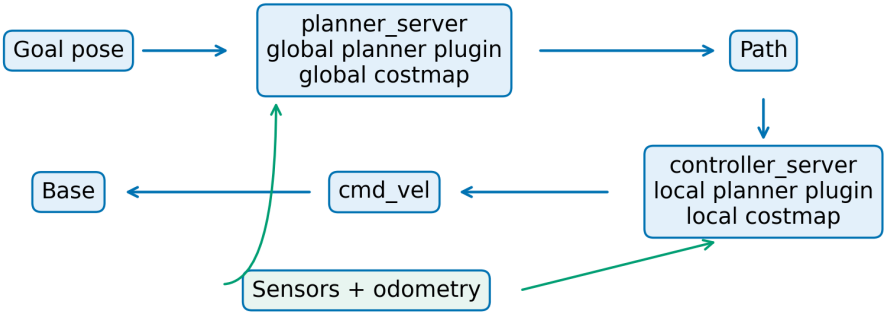
The two missing roles

A **global planner** finds the route around walls. A **local planner** tracks it while reacting to nearby obstacles.

Nav2 local controllers are path trackers with obstacle awareness. Feedback idea: measure error, command (v, ω), repeat. The polar formulas are not used. Default: 20 Hz.

Defaults in the Nav2 repository; check your release.

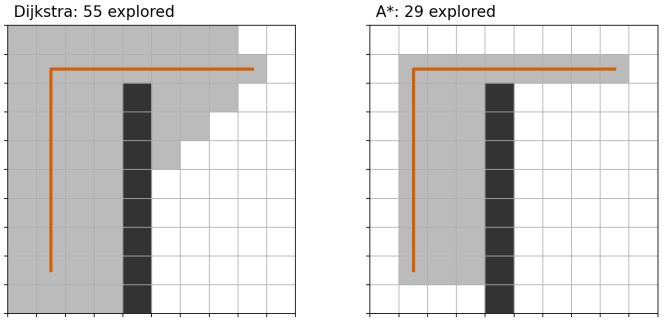
Navigation as two loops: planner server and controller server



Plugin: a replaceable algorithm loaded by name. Default controller: 20 Hz.

ros-navigation/navigation2: ROS 2 navigation stack. Repository defaults; check your release.

Global planner: search on a costmap

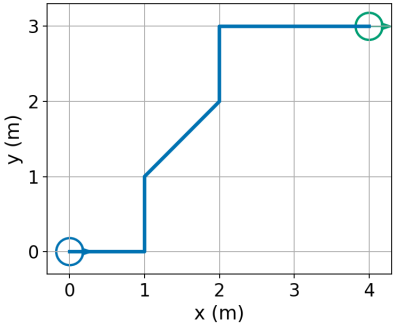


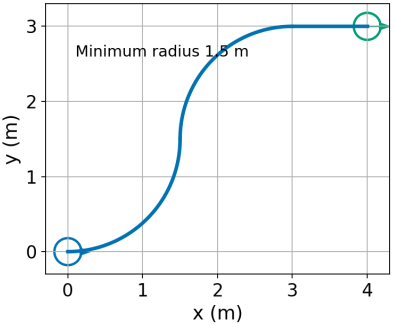
Costmap: danger grid. Input: costmap, start, goal; output: a path of poses.

Dijkstra expands outward; A* uses a heuristic (estimated goal distance) to focus the search.

NavFn: circular robot; default Dijkstra (use_astar=false). Repository defaults; check your release.

Global planners that respect kinematics





Smac 2D: 8-connected grid A* for circular diff./omni.
Hybrid-A*: Dubins/Reeds–Shepp, minimum radius.

Lattice: primitives for diff., omni, legged, Ackermann; any footprint. Feasible search uses actual heading.
Smac package. Illustration, not Nav2 output.

Local planner: follow the path now, avoid what is near

Inputs and output at every control tick

Path + local costmap + odometry ⟶ velocity command (v, ω). The controller server repeats at 20 Hz by default.

Path trackers

Pure pursuit aims at a point on the path. RPP slows near obstacles and sharp curves. README: over 1 kHz on a modern CPU.

Regulated Pure Pursuit

Defaults in the Nav2 repository; check your release.

Sampling planners

DWB and MPPI simulate candidate motions, score them and choose a command.

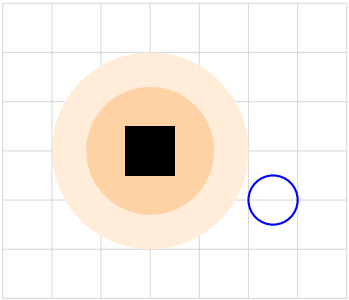
DWB MPPI

Global versus local: the table

Property	Global planner	Local controller
Role	where to go	how to move now
Looks at	whole known map	nearby costmap + odometry
Horizon	whole route	a few seconds
Runs	new goal / replanning trigger	every tick (default 20 Hz)
Kinematics	approximate, or feasible model	configured motion model
Failure	no route / stale map	stuck / oscillation / collision
Examples	NavFn, Smac	DWB, MPPI, RPP

Defaults in the Nav2 repository; check your release.

Costmap and inflation



Obstacle, inflation halo, robot circle

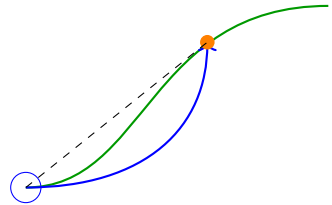
One cost per cell

Values range from free to lethal. **Inflation** spreads obstacle cost outward. Point-cost critics need obstacles expanded by the robot radius, so collision can be assessed at the robot centre.

The global costmap covers the known map. The local costmap is a window around the robot.

Both planners read costmaps; they use them over different distances and times.

Path trackers: from pure pursuit to Regulated Pure Pursuit



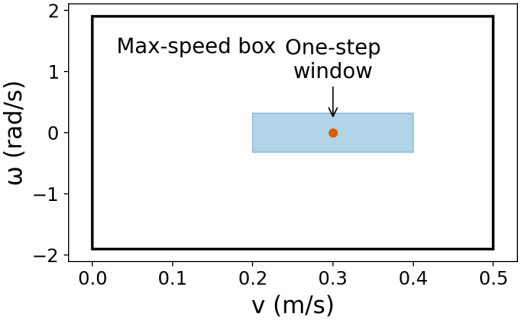
Lookahead point and approach arc

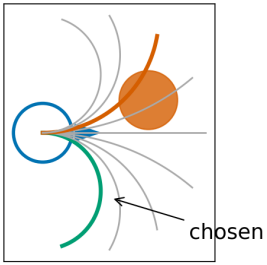
Pure pursuit
Pick a lookahead point on the path; drive an arc toward it. Lookahead distance replaces the three gains, and a path replaces the single goal.

RPP regulates speed near obstacles and sharp curves, and adapts lookahead by speed. Suitable for differential, legged and Ackermann; no omni lateral motion.

RPP package and 2023 paper. README: over 1 kHz on a modern CPU.

DWB: try velocities, simulate, score





1. DWA / LimitedAccelGenerator: one-step acceleration window.
2. Sample (v, ω); default 20 × 20.
3. Simulate constant commands: default 1.7 s.
4. Critic: one scoring function. Select best valid sum.

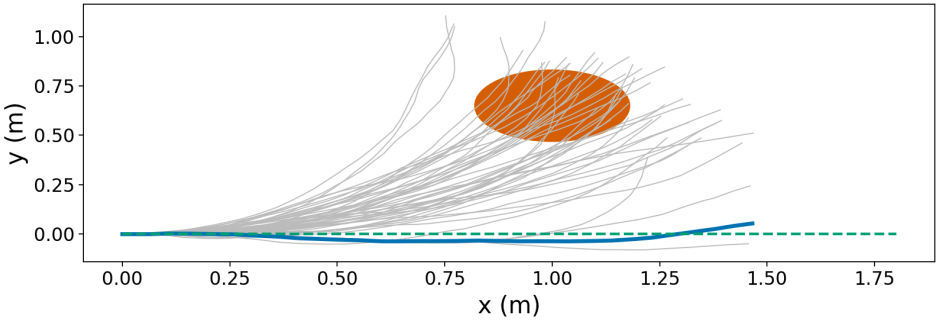
DWB default: StandardTrajectoryGenerator (rollout style). Defaults: check your release.

DWB critics and tuning intuition

Critic	Wants	Too much weight: intuition
BaseObstacle	low costmap cost	stalls in corridors
PathAlign / PathDist	stay on path	refuses detours
GoalAlign / GoalDist	head to goal	cuts corners
RotateToGoal	rotate at goal	creeps
PreferForward	forward motion	escape is harder
Oscillation	no back-and-forth	discourages reversals
Twirling	no spinning (omni)	discourages rotation

Intuition: longer sim_time sees farther, costs more compute and can slow reaction. Critic scales set priorities; collisions stay invalid.

MPPI: sample sequences, weight them, average



1. Shift the last best control sequence.
2. Add Gaussian noise to make a batch.
3. Simulate: diff., omni or Ackermann.
4. Score with critics.
5. Soft-max weights combine controls, not paths.
6. Apply first control; repeat. MPPI package.

Temperature: low is selective; high approaches a plain mean of control sequences.

MPPI parameters and tuning intuition

Parameter	Default	Turn up: intuition
time_steps × model_dt	56 × 0.05 = 2.8 s	farther, more compute
batch_size	1000; bringup 2000	more candidates, CPU
temperature	0.3	less selective weights
vx_std, wz_std	0.2, 0.4	wider perturbations
vx_max, wz_max	0.5 m/s, 1.9 rad/s	faster permitted motion
gamma	0.015	smoothness vs low energy
Critic weights	PathAlign 14; Goal 5; PreferForward 5; Cost 3.81	greater relative priority

2.8 s at 0.5 m/s: 1.4 m lookahead. Visualization slows it down.

Defaults in the Nav2 repository; check your release.

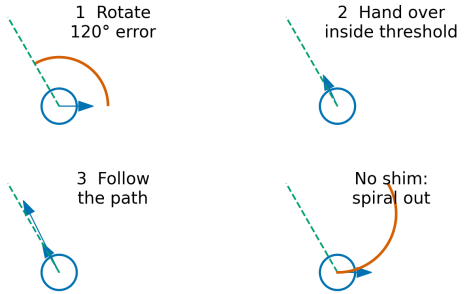
DWB versus MPPI

Property	DWB	MPPI
Samples	DWA: constant velocity pair; default generator: rollout style	noisy control sequences
Combination	select one candidate	soft-max weighted controls
Models	differential / omni generators	differential / omni / Ackermann
Reported speed	not established here	50+ Hz, 4th-gen i5 README
Tuning	critic scales, samples, horizon	weights, noise, temperature, horizon
Bringup default	earlier controller	current parameter file

RPP is useful when a good path needs smooth tracking. Reported performance is hardware-specific.

Defaults in the Nav2 repository; check your release.

The rotation shim: turn first, then drive



Turn, then delegate

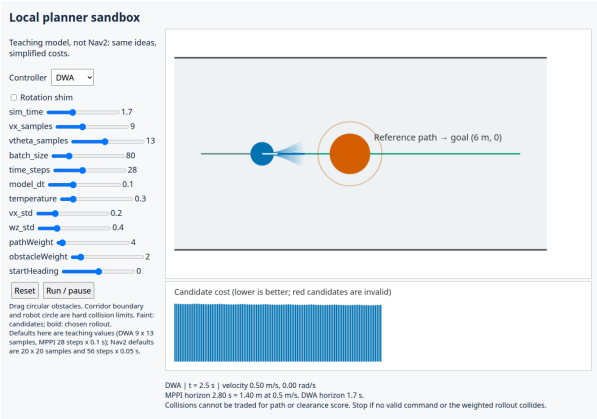
DWB/TEB can spiral out from large heading errors. The shim rotates, then delegates.

Engage 0.785 rad; hand over at 0.3925 rad.
Rotation 1.8 rad/s; maximum acceleration 3.2 rad/s^2.
Heading sampled 0.5 m ahead.
Collision horizon 1.0 s; return control if blocked.

Rotation shim package.

Repository defaults; check your release.

Demo 2: three controllers on the same path



Teaching model, not Nav2. The demo uses smaller sample counts and a shorter horizon than the Nav2 defaults shown earlier.

Choosing a stack for a base, and what to remember

Base	Global	Local	Rotation shim
Differential	Smac 2D / NavFn	MPPI / DWB / RPP	with non-feasible planner
Omni	Smac 2D / NavFn	MPPI / DWB	useful
Ackermann	Hybrid / Lattice	MPPI Ackermann / RPP	no in-place rotation

Five ideas to remember

Constraints determine mobility.

Holonomic test: DDOF = DOF.

Polar feedback: $k_\rho > 0$, $k_\alpha > k_\rho$, $k_\beta < 0$.

Global: route; local: next few seconds.

A rotation shim turns first when the base can rotate in place.

Reference Materials

- Siegwart, Nourbakhsh, Scaramuzza — Siegwart Nourbakhsh Scaramuzza Introduction to Autonomous Mobile Robots Second Edition
- Macenski et al. — Nav2 navigation stack, ros-navigation/navigation2
- Macenski, Singh, Martin, Gines — Regulated Pure Pursuit for Robot Path Tracking
- Fox, Burgard, Thrun — The dynamic window approach to collision avoidance
- Nav2 contributors — MPPI controller, nav2_mppi_controller README

Deserve “A” grade!

– Oleg Bulichev

bulichev.ov@mipt.ru

@Lupasic

Цифра, 521